

THE CURIOSITY CUP 2024
A Global SAS® Student Competition

Ham, Spam and Phishing Mails Classification

Team DataTitans

ABSTRACT

In the rapidly evolving domain of cybersecurity, differentiating between legitimate emails (Ham), unwanted advertising or malicious emails (Spam), and fraudulent emails attempting to deceive recipients into divulging sensitive information like credentials and personal details (Phishing) is paramount. With phishing incidents alone causing over \$4.2 billion in losses in the US in 2020 and engaging up to 30% of recipients, the magnitude of their threat is undeniable. This research addresses the challenge of identifying these malicious emails by employing machine learning techniques on a dataset reflective of real-world scenarios.

Our objective is to develop a model that accurately flags spam and phishing attempts to enhance user security and reduce cyber threats. Initially relying on basic textual representation & analysis like TF-IDF, our approach evolved to incorporate advanced contextual embeddings and custom features inspired by cybersecurity best practices. This paper details our innovative methodology, findings, and the iterative refinement of our model, aiming to set new standards for email classification. All the work is done using SAS Viya for Learners and Python.

PROBLEM

We tackle a supervised classification problem with text data in email format, categorized as an NLP (Natural Language Processing) task. Our objective is to classify emails into legitimate ham mail, non-legitimate spam, or malicious phishing mail.

DATA

The dataset comprises 11 datasets spanning from 1998 to 2022, sourced from publicly available datasets pivotal in cybersecurity and machine learning research. Two datasets exclusively contain phishing emails, one exclusively contains spam emails, and four contain a mix of spam and ham emails. The remaining four datasets are redundant or unreadable. We started by removing missing values and duplicates, filtering non-string data, and removing emails in languages other than English for simplicity. Then we used The SAS Studio's Text Topics tool to highlight insights from email subjects (Figure 1). Legitimate emails focus on technical topics, while spam and phishing emails prioritize enticing and deceptive subject lines. This may skew the dataset towards specific professional sectors like IT, potentially limiting its applicability to real-life scenarios. We choose precision as the evaluation metric due to the serious risks associated with misclassifying phishing mails.



Figure 1. Text Topics included in mails grouped by class

METHODOLOGY

Our iterative methodology starts with insight-driven exploration, followed by result analysis, idea generation for addressing limitations, and iterative refinement.

APPROACH 1: BENCHMARK

Data Preparation

We establish a benchmark model, which is a good practice in machine learning projects. With text data, we need vectorization for machine interpretation, which necessitates a rigorous cleaning procedure. We start by keeping only alphanumeric characters. Numerical values are important, especially in monetary contexts. However, because dates are common in emails and their numerical values could bias the analysis, we treat them separately by replacing them with "date". Next, we convert the text to lowercase, remove stop words, and perform lemmatization to retain only impactful words. These optimizations are crucial, especially for large vocabularies, as TF-IDF processing can be slow. Therefore, a cleaned version of a text like "Hello Aymane! See you on Friday 02/02/2024 at 22 pm. Best, Chadha" becomes "hello aymane see friday date 22 pm best chadha". Then, we use a statistical method to analyze word distribution in emails. We apply percentiles to exclude outliers, removing emails with excessively long bodies or very few words, which may disrupt accurate classification.

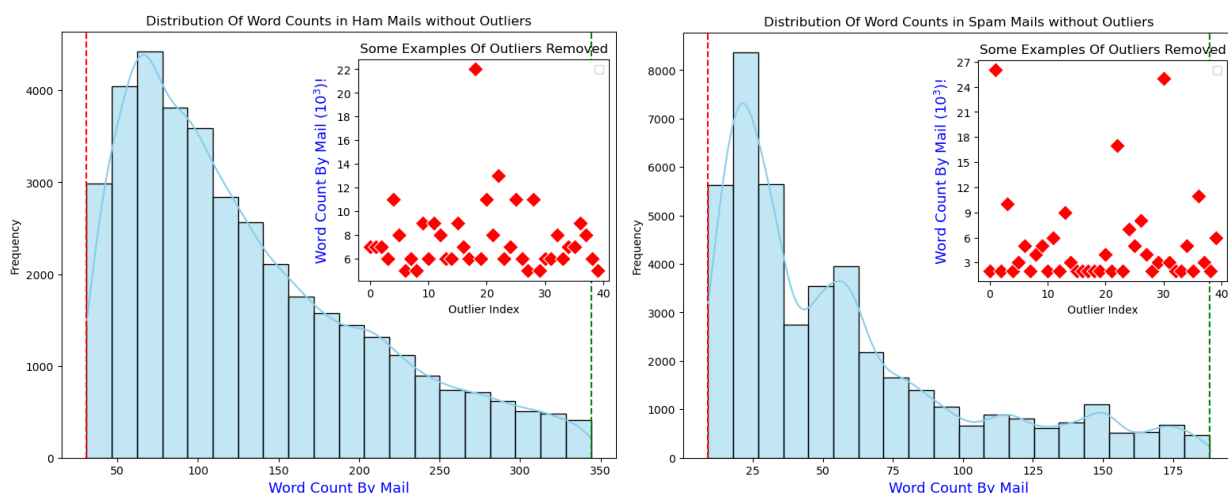


Figure 2: Word Count Distribution and Anomalies in Ham and Spam Emails

Emails are split into two sets: 'ham_spam_phishing_train_val' contains 80% of the cleaned data (17,530), ensuring a balanced split between train and test by selecting 80% from each class. It includes 'Body,' 'Subject,' and a 'label' categorizing emails (0 for ham, 1 for spam, 2 for phishing). The remaining 20% emails (4,384) form the 'ham_spam_phishing_test' set, with the split remaining consistent throughout the study for fair comparisons. To gain better insights, we plotted word count by class and observed that phishing emails tend to be longer (Figure 3 Appendix).

Data Analysis

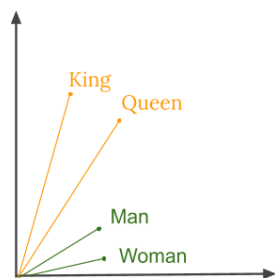
First, we transform the text into a format understandable by the model, a process known as vectorization. We opted for TF-IDF (Term Frequency-Inverse Document Frequency), a widely used technique in NLP tasks, it represents the importance of a word in a document relative to a collection of documents, highlighting words that are frequent in a document but rare across documents. We set a maximum of 100 features. As this first approach serves as

a benchmark, we used a simple logistic regression model, which surprisingly yielded excellent results! It achieved on test data a Precision of 90.91% and an F1 score of 90.97%. Out of curiosity, we tested the model on some legitimate emails from our inbox, only to discover an unexpected outcome (Figure 4 Appendix). None of the phishing emails were well classified. Further examination revealed that the model exhibited hypersensitivity to certain words like 'please' and 'available', incorrectly categorizing them as spam. This discovery marked the first limitation we encountered.

Results Analysis and Reflection

The model performs well with test data, meeting our training objectives. However, it shows high sensitivity to some words, likely due to the vectorization method. We plan to explore alternatives. Additionally, discrepancies in data distribution between training and new emails could impact performance, (e.g. topics diverge from IT to medicine).

APPROACH 2: WORD EMBEDDING



In NLP, word embedding represents words as vectors in a continuous space, capturing semantic relationships and contextual meaning by measuring similarity and distances between word vectors. Closer word vectors signify similar meanings. Unlike TF-IDF, which prioritizes word frequency and importance within documents.

Figure 5: Word Embedding Example: Similar Angle Between 'Man' and 'Woman' as 'King' and 'Queen', but Different Lengths Reflecting Topic Variance

Data Preparation and analysis

Adapting the cleaning process to the word embedding approach involves retaining certain elements, such as interrogation or exclamation symbols, which may carry contextual nuances important for understanding. Unlike lemmatization, which simplifies word forms, we opt to preserve different word forms to capture varied meanings or connotations. We maintain the same data split and cleaning steps as in the previous approach. Leveraging the "en_core_web_lg" vocabulary library from spaCy, which offers extensive text data-trained, large-sized word vectorization (we used 300), enhances the model's capability to discern and interpret linguistic nuances, beneficial for tasks requiring contextual word analysis.

After testing various models and tuning hyperparameters, logistic regression achieves the highest precision of 95.80%, slightly surpassing SVM and with a 4% advantage over random forest, indicating a potential linear separation boundary in the data. Moreover, it exceeds the benchmark by 5%, an improvement that affirms the efficacy of word embedding in capturing intricate text details. Upon analyzing the 6 misclassified phishing emails as ham, we found them challenging to distinguish due to deceptive content (e.g., "dear user, windows server update. account required update avoid closed used 99% email capacity please click unlock messages update account..."). Also, ambiguity persists in personal data between spam and phishing (Figure 4 Appendix), prompting further consideration of the model's generalization ability and the overall vectorization approach!

Results Analysis and Reflection

To overcome this challenge, we pivot our approach, leveraging human expertise and cybersecurity guidelines to engineer features directly from the original email content. This process, known as feature engineering, allows us to incorporate domain knowledge and insights into the model, potentially improving its performance and adaptability.

APPROACH 3: FEATURE ENGINEERING

Often, you don't need to read an entire email to discern whether the sender is seeking a response, expressing excitement, or conveying urgency. Recognizing specific symbols or keywords associated with certain themes can suffice to understand the sender's intent

Feature Extraction

We used SAS code in SAS Viya to extract relevant information about symbols and some words used in the body. First, we created a new column "mail" resulting from the combination of columns "body" and "Subject", as this new column will be more informative. We then extracted the following features from the body "mail" and stored results in new columns:

- Number of exclamation marks: Indicator of the sender's excitement or urgency.
- Number of question marks: Presence of inquiries or requests.
- Number of digits: Quantitative measure offering insights into the numerical content.
- Number of special characters: This metric assesses the use of specific symbols ('+', '@', '#', etc.) in emails, offering insights into communication style and intent.
- Number of Thanking Words and Phrases: Through identifying prevalent expressions of gratitude (e.g., 'thank you', 'much appreciated'), this measure quantitatively assesses the level of politeness or gratitude conveyed in the communication.
- Number of Request Words and Phrases: Detection of commonly used request-related expressions (e.g., 'please', 'I need'), highlighting the transactional elements.

For the enumeration of symbols, the *countc* function in SAS was utilized. Besides, The SAS code for counting the number of thank-you words and phrases is provided in Appendix Document 1.

Furthermore, an examination was conducted to ascertain the presence of email addresses within the 'mail' column and to determine whether each email explicitly requests a reply from the recipient. This involved three main steps: first, spotting email addresses in the email content; second, extracting text around the email address, up to 300 characters on both sides; and third, looking for phrases in this text that indicate a request for a reply, like "respond" or "get back to me."

These extracted features align with cybersecurity guidelines that emphasize vigilance against emotionally charged or urgent communication, scrutiny of suspicious inquiries, and awareness of social engineering tactics.

Data Analysis

Our data study reveals distinct patterns in the use of special characters across 'ham', 'spam', and 'phishing' emails. In Appendix Figure 5, you'll notice that 'ham' emails often feature a higher ratio of special characters, suggesting diverse and expressive language use. In contrast, 'phishing' and 'spam' emails generally show restraint in special character use, likely a tactic to evade spam filters and appear more credible. In the second pie chart of the same figure, you'll observe the percentage of emails containing email addresses and whether a reply is solicited in each class. Notably, a significant proportion of phishing emails (28.1%) seek a response, a tactic often used to extract sensitive information or draw recipients into deceptive exchanges. In contrast, a smaller percentage of spam emails (7.0%) appear to invite replies, possibly to engage you further or to verify the activity of your email account.

Further insights from the data suggest that 'spam' and 'phishing' emails typically forgo interrogatives, preferring direct statements that incite action. Additionally, 'phishing' emails may sparingly use exclamation marks to reduce suspicion, while 'spam' may use them more freely to create urgency, aiming to engage you, the receiver.

Data Modeling and results

Our aim was to develop a pipeline for testing multiple models and selecting the most effective one (Figure 6 Appendix). To ensure our model's independence from specific features like "Subject," "Source," and "Body," we rejected them and designated our "Label" as the target variable. The "Mail" column was designated as Text. Our pipeline architecture comprises three branches. The first branch started with a SAS code node implementing Weight of Evidence Encoding (WOEENCODE) to translate variables into a numeric format. Then, a transformation node identified numerical or binning transformations, enhancing model performance, followed by a Bayesian network model. The second branch contained a Feature Machine node creating new features based on data patterns, followed by a variable selection node that reduces the number of features. The final branch featured a Data Mining node extracting categories and identifying topics within email content, followed by a transformation's node as well as Gradient Boosting and Neural Network models. We assessed model performance with and without the data mining component, to evaluate the added value of the previously defined features. After hyperparameter tuning, we ranked models based on their Precision scores. Gradient Boosting, from the Text Mining branch, emerged as the champion model with an impressive score of 98.28% on the test set, followed by the Neural Network at 94.52%, and a secondary Gradient Boosting model at 87.36% in the first branch. These results highlight the independent contribution of previously extracted features to strong model performance, indicating that significant information within our emails resides in the symbols and some keywords utilized. After testing the model on our personal mail dataset, the confusion matrix (Figure 4 Appendix) revealed a challenge in differentiating between spam and phishing emails, with phishing emails often mislabeled as spam. However, the model excels in correctly flagging all phishing emails as malicious, a crucial function for effective email security systems.

FURTHER STUDIES

Hard Generalization Process: Distinguishing between phishing and spam in an individual's inbox may seem straightforward to the vigilant eye, yet creating a universally applicable model for personalized classification poses inherent challenges. We suggest the development of models that, rather than solely analyzing email content, incorporate additional inbox data such as email receipt frequency, sender addresses, domain information, localization, and file extension analysis. Addressing privacy concerns is crucial, for which we propose leveraging Federated Learning—a decentralized approach respecting user privacy by design. Furthermore, we suggest integrating Reinforcement Human Feedback mechanisms, enabling the model to seek user or expert confirmation on uncertain classifications.

Advanced Email Content Understanding: Conventional methods often miss the nuanced aspects of email content. We advocate for Large Language Models (LLMs), adept at detecting manipulative language and implicit cues. Additionally, transitioning to a graph-based email representation offers promise. This approach involves structuring emails as graphs, with nodes representing elements like headers, body text, and signatures, and entities such as individuals, locations, and dates. This strategy aims to emulate human perception of emails, since we naturally interpret them as interconnected layers guided by our senses and logic, and not just as text or strings of words!

CONCLUSION

In conclusion, our proposed methodologies and technological innovations hold great promise for advancing email classification, providing robust protection against evolving spam and phishing threats. However, some limitations persist, and further exploration remains necessary.

REFERENCES

The national Cyber Security Centre, NCSC, UK, "Phishing: Spot and Report Scam Emails, Texts, Websites and Calls". Accessed January 25.

<https://www.ncsc.gov.uk/collection/phishing-scams/spot-scams>

Zenodo. Sep 13 2023. "Phishing Email Curated Datasets". DOI [10.5281/zenodo.8339690](https://doi.org/10.5281/zenodo.8339690) . Available at <https://zenodo.org/records/8339691>

SAS Institute Inc., SAS Campus Drive, Cary, North Carolina 27513, USA. "SAS Visual Text Analytics". Accessed February 5. <https://video.sas.com/category/videos/sas-visual-text-analytics>

IBM. "What is Federated Learning?". 24 Aug 2022. Accessed February 16. Available at <https://research.ibm.com/blog/what-is-federated-learning>

Minkov, Einat. March 2012. "Adaptive Graph Walk Based Similarity Measures in Entity-Relation Graphs". Available at https://www.researchgate.net/figure/A-joint-graph-representation-of-email-and-meetings-data_fig4_238111746

APPENDIX

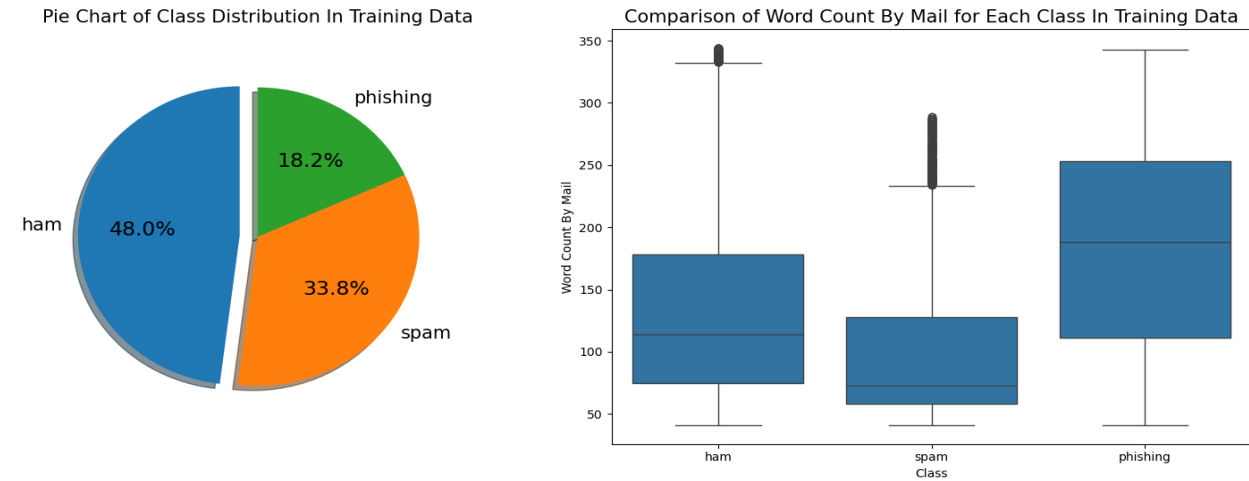


Figure 3: Class Distribution Pie Chart and Word Count Comparison in Train Data

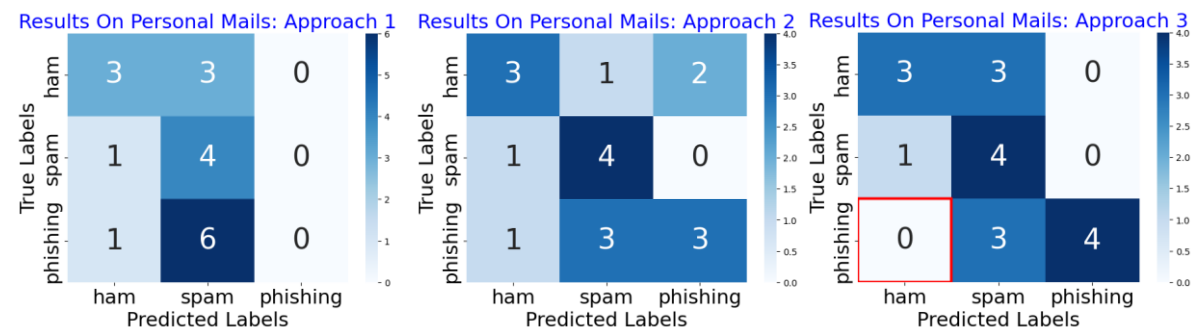


Figure 4: Confusion Matrix for Best Models in Approach 1, 2 & 3 on Personal Data

```
data OUTPUTDATA;
set WORK.INPUT_DATA;
array request_phrases[17] $50 _temporary_ ( 'please', 'kindly', 'could',
'would you', 'can you', 'help me', 'assist me', 'I need', 'please assist',
'I request', 'I require', "I'm requesting", "I'm in need of", "I'm looking
for", "I'm seeking", 'I would like', 'I would appreciate');
requests_count = 0;
do i = 1 to dim(request_phrases);
    patID = prxparse(cats('/(?i)', request_phrases[i], '/'));
    start = 1; /* Initialize starting position for regex search */
    do while(start <= length(mail));
        posit _ ion = prxmatch(patID, substr(mail, start));
        if position > 0 then do;
            requests_count + 1;
            start = start + position - 1 + prxposn(patID, 0, length);
        end;
    else leave;
    end;
    call prxfree(patID); /* Free Memory */
end;
run;
```

Document 1: SAS Code for counting thanking words

Nodes	Parameters
Bayesian Network	Number of bins: 10; Network structure: Naive, Tree Augmented and Parent-Child; Maximum Parents: 5; Parenting method: Set of parents.
Gradient boosting	Number of trees: 100; Learning rate: 0.05; L2 regularization: 1; Interval Target distribution: Normal
Neural Network	Input Standardization: Midrange; Number of hidden Layers : 5; Number of neurons per layer: 50; Hidden layer activation function: Tanh
Text Mining	Parsing options: Include parts of Speech, Extract noun groups and Extract entities; Minimum number of documents: 4
Feature Machine	Transformation Policy: Cardinality, Missingness and Skewness; Input Variable Screening: Coefficient of Variation and Group rare levels; Leakage percentage threshold: 90
Variable selection	Correlation statistics: correlation; Cumulative variance cutoff: 1; Incremental variance cutoff: 0.001; perform sequential selection

Table: Parameters used in each node in the building stage

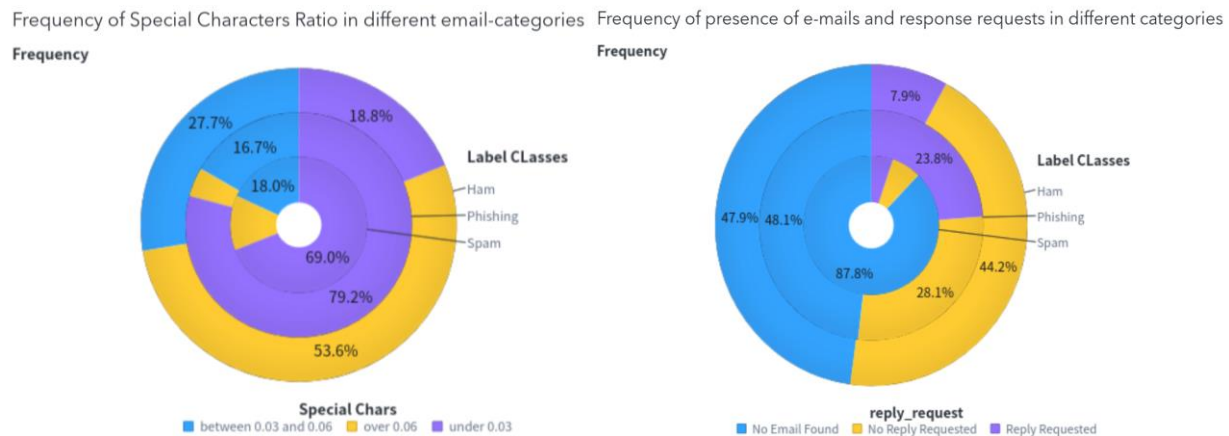


Figure 5: Special Characters Ratio, Email Presence & Reply Request By Class

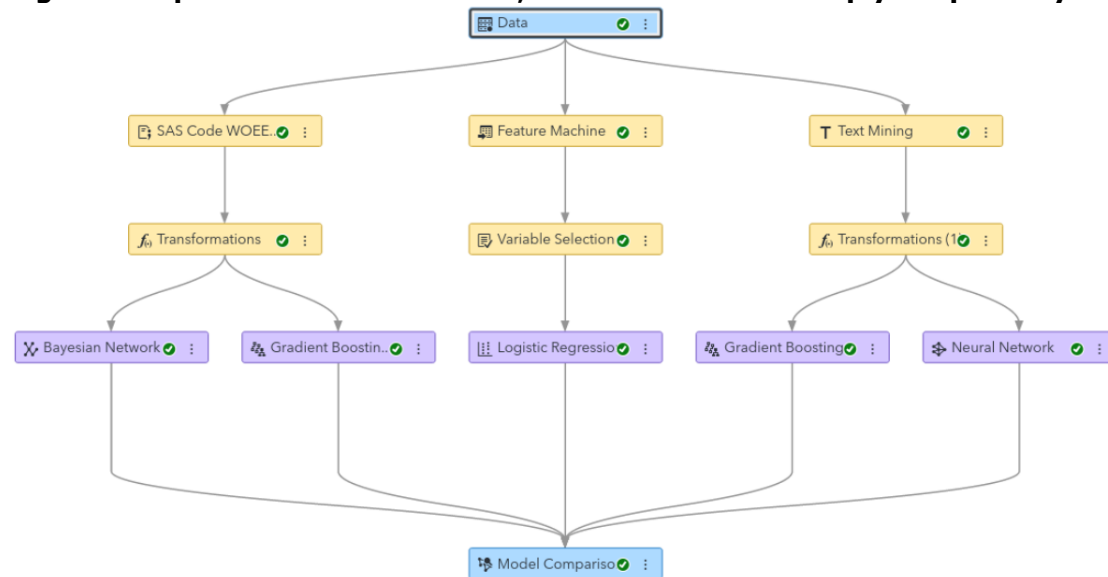


Figure 6: Pipeline used in Approach 3 in SAS Viya